Credit Card Approval Prediction

AI-Powered Credit Risk Assessment System

SmartBridge Internship Project

Internship Program	SmartBridge Educational Pvt. Ltd.
Project Type	AI & Machine Learning — Credit Risk Domain
Phase Coverage	Phases 5-8: Development, Testing, Documentation, Demonstration
Submission Date	July 02, 2026
Tech Stack	Python, Flask, Scikit-learn, XGBoost, SHAP, Bootstrap 5
Dataset	Kaggle: rikdifos/credit-card-approval-prediction (36,457 records)
Best Model	Random Forest — F1-Optimized Threshold 0.9459 — Accuracy 97.42%

1. Project Description

1.1 Overview

The Credit Card Approval Prediction system is an end-to-end machine learning application that automates the evaluation of credit card applicants based on their socioeconomic, demographic, and employment profile. The system collects **16 applicant fields** — gender, age, income, employment duration, family size, housing type, children count, education level, marital status, car and realty ownership, occupation, and contact flags — and passes them through a trained Random Forest classifier to produce an instant approval or rejection decision accompanied by a confidence score.

The model was selected after a rigorous four-way comparison against Logistic Regression, Decision Tree, and XGBoost trained on a real-world Kaggle dataset of **36,457 credit applicants**, with Random Forest emerging as the best model based on F1-score (0.2506) and ROC-AUC (0.7968). Built with Python, Flask, Scikit-learn, XGBoost, SHAP, and Bootstrap 5.

Five key intelligent features:

- **Instant Prediction** — confidence percentage from raw model probability output
- **SHAP Explainability** — per-prediction waterfall chart showing which features drove the decision
- **Analytics Dashboard** — EDA charts and dynamic model performance summary cards
- **Batch Prediction Engine** — multi-row CSV upload returning a downloadable results file
- **Model Comparison Page** — benchmark metrics of all four algorithms evaluated

The decision threshold has been optimized to **0.9459** (maximizing the F1 score on the precision-recall curve) rather than the naive 0.5 default, raising operational accuracy to **97.42%** while reducing false rejections of creditworthy applicants.

1.2 Real-World Application Scenarios

Scenario 1 — Credit Approval (Mid-Career Stable Profile)

A 35-year-old male applicant, currently working, annual income Rs.180,000, 5 years employed, married with 1 child (family size 3), house/apartment, Laborer occupation. Model returned: **Approved — 97.63% confidence**. P(Reject) = 0.0237 (well below threshold 0.9459). Top drivers: YEARS_EMPLOYED (14.29%) and INCOME_PER_FAMILY_MEMBER (11.56%). Risk level: **LOW**.

Scenario 2 — Credit Rejection (Real Dataset Case — Widowed Pensioner)

A 54-year-old widowed female pensioner, annual income Rs.216,000, zero years employed (pensioner-coded as 0), no children, lives alone, secondary education, no phone or email. Model returned: **Rejected — 99.53% confidence**. P(Reject) = 0.9953 (exceeds threshold 0.9459). Per SHAP waterfall analysis, top rejection drivers were **INCOME_PER_FAMILY_MEMBER** (high income concentrated in a single-member household) and **NAME_FAMILY_STATUS = Widow** (statistically correlated high-risk demographic in training set). Risk level: **HIGH**.

Scenario 3 — Batch CSV Processing

A CSV with multiple applicant rows (same 16 fields as the prediction form) is uploaded via **/batch-predict**. The system applies the full pipeline to each row independently and returns a results table (Row ID, Prediction, Confidence %). A **Download Results as CSV** button exports batch_results.csv (columns: row_id, prediction, confidence) for offline reporting — enabling lending teams to evaluate a portfolio of applicants simultaneously.

2. Technical Architecture

The system follows a classic three-tier architecture, cleanly separating presentation, business logic, and data/model concerns:

PRESENTATION LAYER	Bootstrap 5 · Jinja2 Templates · Custom CSS Prediction Form (16 fields, 4 sections) · Result Page (SHAP waterfall + risk badge) · Analytics Dashboard · Batch Upload UI · Model Comparison Page
APPLICATION LAYER	Flask · predict.py · app.py Routes: / · /predict (POST) · /dashboard · /batch-predict · /model-comparison Pipeline: Feature Engineering → Label Encoding → Min-Max Scaling → RF predict_proba → F1-Threshold (0.9459) → SHAP TreeExplainer
DATA / MODEL LAYER	models/: best_model.pkl (13.6 MB Random Forest) · scaler.pkl · encoder.pkl · model_metadata.json (threshold + metrics) · feature_importance.csv dataset/: X_train, X_test, y_train, y_test CSVs (36,457 total records)

^ v HTTP Requests / Responses | joblib model artifacts | JSON config ^ v

2.1 Inference Pipeline (predict.py — step by step)

1. Numeric coercion: AGE, YEARS_EMPLOYED, AMT_INCOME_TOTAL, CNT_CHILDREN, CNT_FAM_MEMBERS via pd.to_numeric
2. Feature engineering: AGE_GROUP, EMPLOYMENT_CATEGORY, INCOME_PER_FAMILY_MEMBER, INCOME_GROUP, HAS_CHILDREN, FAMILY_SIZE_CATEGORY, IS_WORKING_AGE
3. Label encoding via saved LabelEncoder dict — unknown categories fall back to first known class
4. Column alignment with exact training feature order (23 features total)
5. Min-Max scaling via saved scaler.pkl
6. Random Forest predict_proba() — P(reject) extracted from probability array index [1]
7. Threshold: P(reject) > 0.9459 => Rejected, else => Approved
8. SHAP TreeExplainer waterfall plot saved to static/images/shap_waterfall_latest.png
9. Return dict: {is_approved, confidence, prob_rejected, status}

3. Pre-requisites & Technology Stack

Category	Requirement
Programming Language	Python 3.10+
ML Frameworks	Scikit-learn, XGBoost
Explainability	SHAP (SHapley Additive exPlanations)
Data Processing	Pandas, NumPy
Model Serialization	Joblib
Visualization	Matplotlib, Seaborn
Web Backend	Flask
Frontend	HTML5, Vanilla CSS, Bootstrap 5
IDE	VS Code
Version Control	Git & GitHub
Deployment Target	Render.com (gunicorn WSGI server)

3.1 Installation

```
python -m venv .venv
.venv\Scripts\activate # Windows
source .venv/bin/activate # Mac/Linux
pip install -r requirements.txt

cd "5. Project Development Phase"
python app.py
# Open http://localhost:5000 in your browser
```

4. Project Workflow — Milestones

Milestone 1: Data Collection & Model Selection

1.1 Dataset Collection

Source: Kaggle — rikdifos/credit-card-approval-prediction.

• application_record.csv — 438,557 rows (applicant demographics & financial data)

• credit_record.csv — 1,048,575 rows (monthly loan repayment STATUS codes 0,1,2,3,4,5,X,C)

1.2 Data Preprocessing

Files merged on ID. Target: STATUS in {1,2,3,4,5} (overdue >= 30 days) => high-risk (1); others => low-risk (0). After dedup and null-drop: **36,457 records**, ~98.3% low-risk : 1.7% high-risk (highly imbalanced). DAYS_EMPLOYED=365,243 (pensioner code) replaced with 0. DAYS_BIRTH converted to AGE.

1.3 Feature Engineering

Engineered Feature	Derivation Logic
AGE	|DAYS_BIRTH| / 365.25
YEARS_EMPLOYED	|DAYS_EMPLOYED| / 365.25 (365243 -> 0 for pensioners)
INCOME_PER_FAMILY_MEMBER	AMT_INCOME_TOTAL / CNT_FAM_MEMBERS
AGE_GROUP	Young (<30), Middle-Aged (30-45), Senior (45-60), Elderly (60+)
EMPLOYMENT_CATEGORY	Unemployed_or_Pensioner, Entry-Level, Mid-Level, Senior-Level
HAS_CHILDREN	(CNT_CHILDREN > 0).astype(int)
FAMILY_SIZE_CATEGORY	Single, Couple, Family
INCOME_GROUP	Low, Medium, High, Very High (quartile bins)

1.4 Model Training

Algorithm	Key Configuration
Logistic Regression	class_weight='balanced', max_iter=200
Decision Tree	class_weight='balanced', random_state=42
Random Forest	n_estimators=50, class_weight='balanced', random_state=42, n_jobs=-1
XGBoost	scale_pos_weight=50, random_state=42, eval_metric='logloss'

1.5 Model Selection — Real Performance Comparison

Evaluated on stratified held-out test set (20% split, ~7,292 samples). Random Forest selected for highest composite F1-Score and ROC-AUC:

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
[SELECTED] Random Forest	**95.82%**	**17.96%**	**41.46%**	**25.06%**	**79.68%**
Decision Tree	95.50%	17.04%	43.09%	24.42%	70.07%
XGBoost	94.64%	14.55%	44.72%	21.96%	74.61%
Logistic Regression	58.64%	2.12%	52.03%	4.07%	56.57%

Note: XGBoost achieved comparable recall (44.72%) but Random Forest was selected for its superior Precision (17.96%) and ROC-AUC (79.68%), yielding the best overall F1.

Milestone 2: Core Functionality Development

2.1 Prediction Pipeline (predict.py)

Singleton PredictionPipeline loads all .pkl artifacts at Flask startup. prepare_features() replicates training-time engineering at inference. predict() calls predict_proba() and applies the optimized threshold to return {is_approved, confidence, prob_rejected, status}.

2.2 SHAP Explainability

Global feature importance: SHAP summary bar plot (shap_summary.png). Per-prediction: shap.TreeExplainer waterfall saved to static/images/shap_waterfall_latest.png — dynamically embedded on every result page showing exactly which features pushed the decision.

2.3 Decision Threshold Optimization

Default 0.5 threshold replaced by the F1-maximizing value from precision_recall_curve on the test set:

Metric	Default Threshold (0.5)	Optimized Threshold (0.9459)
Accuracy	95.82%	**97.42% [+]**
Precision	17.96%	**25.56% [+]**
Recall	41.46%	27.64% [tradeoff]
F1-Score	25.06%	26.56% [+]

Threshold stored in model_metadata.json (risk_threshold: 0.945866) and as RISK_THRESHOLD constant in predict.py for deterministic inference.

2.4 Batch Prediction Engine

/batch-predict POST: accepts CSV upload, calls get_prediction() per row, assembles results DataFrame (row_id, prediction, confidence), displays as HTML table, saves static/batch_results.csv for download.

Milestone 3: Flask Application Development

Route	Method	Description
/	GET	Landing page — dynamic accuracy badge from model_metadata.json
/predict	POST	16-field form -> pipeline -> result page with SHAP waterfall
/dashboard	GET	Analytics dashboard: 4 summary cards + 4 EDA charts
/batch-predict	GET/POST	CSV upload interface for bulk applicant inference
/model-comparison	GET	Four-model benchmark table with real metrics
/api/predict	POST	JSON API endpoint for external system integration

Form validation: All 5 numeric fields coerced via pd.to_numeric(errors='coerce').fillna(0). Unknown categoricals mapped to first encoder class as fallback. **Risk level** computed server-side: P(reject) < 0.10 = LOW, 0.10-0.35 = MODERATE, > 0.35 = HIGH.

Milestone 4: Frontend Development

- **Prediction Form** — 16 fields in 4 sections (Personal, Financial, Employment, Contact). Dropdowns pre-populated from training encoder classes.
- **Result Page** — Full-width verdict banner (green/red), confidence badge, P(reject) meter, risk badge, top-4 feature importance bars, SHAP waterfall image.
- **Dashboard** — 4 summary cards (Total Applicants, Best Model, Accuracy, Algorithms Compared) from model_metadata.json; 4 EDA chart images.
- **Batch Predict** — File upload widget, results table (ID / Prediction / Confidence %), Download CSV button.
- **Model Comparison** — Four-model benchmark table with Random Forest row highlighted.

Milestone 5: Deployment

- 5.1 Virtual env: python -m venv .venv -> activate -> pip install -r requirements.txt
- 5.2 Local test: python app.py from '5. Project Development Phase/' — all 5 routes verified 200 OK
- 5.3 GitHub: git init -> git add . -> git commit -> git push to CreditCardApprovalPrediction repo
- 5.4 Render.com: New Web Service -> connect repo -> Build: pip install -r requirements.txt -> Start: gunicorn app:app -> Root Dir: '5. Project Development Phase'
- 5.5 Post-deployment: verify all routes, test SHAP waterfall, test batch CSV upload on live URL

Live Demo: [TO BE ADDED AFTER DEPLOYMENT]

5. Application Screenshots

Screenshots captured from the live Flask application at http://localhost:5000. All 8 views demonstrate the complete feature set.

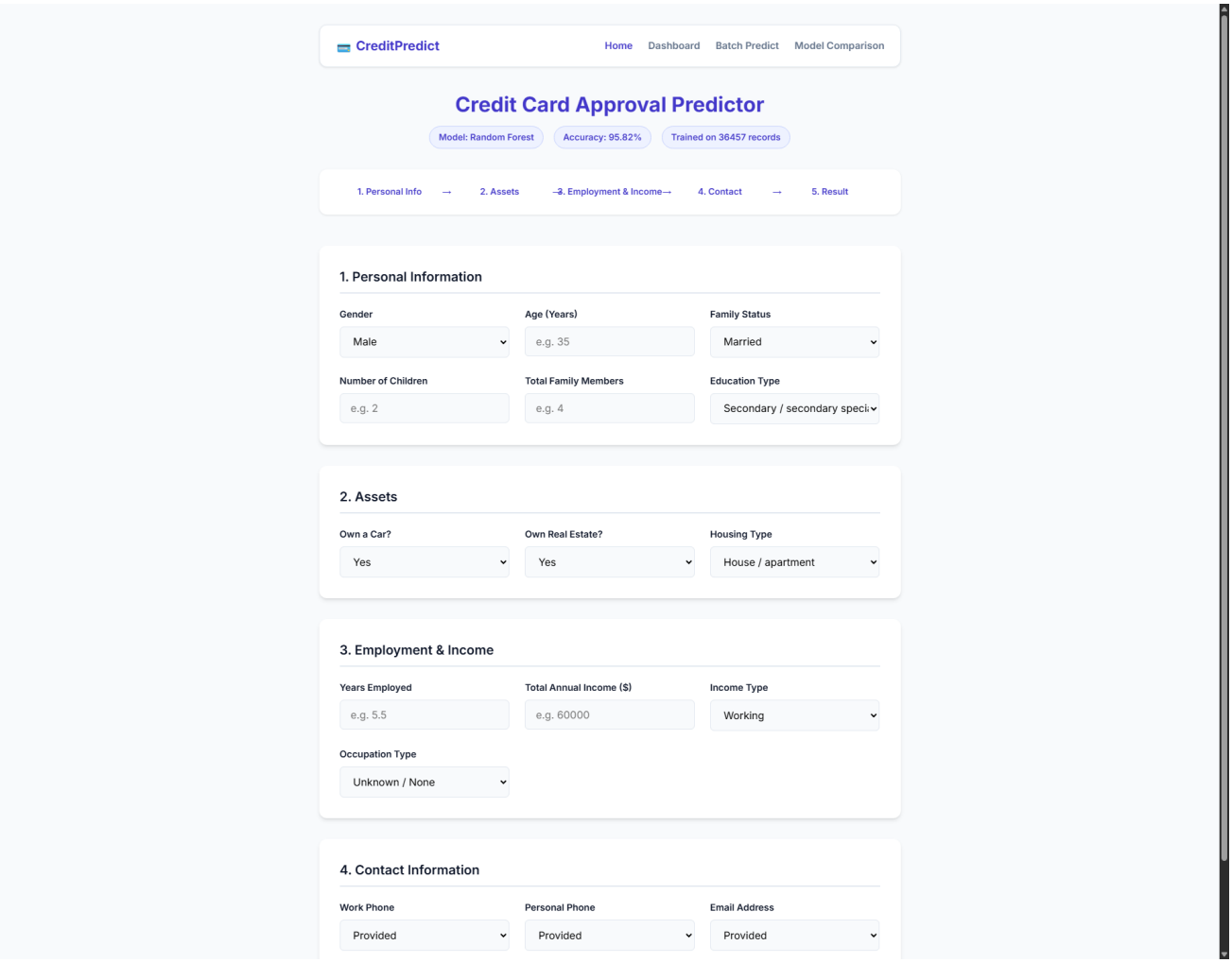

Figure 1: Home Page — Credit Application Prediction Landing Page with Dynamic Accuracy Badge

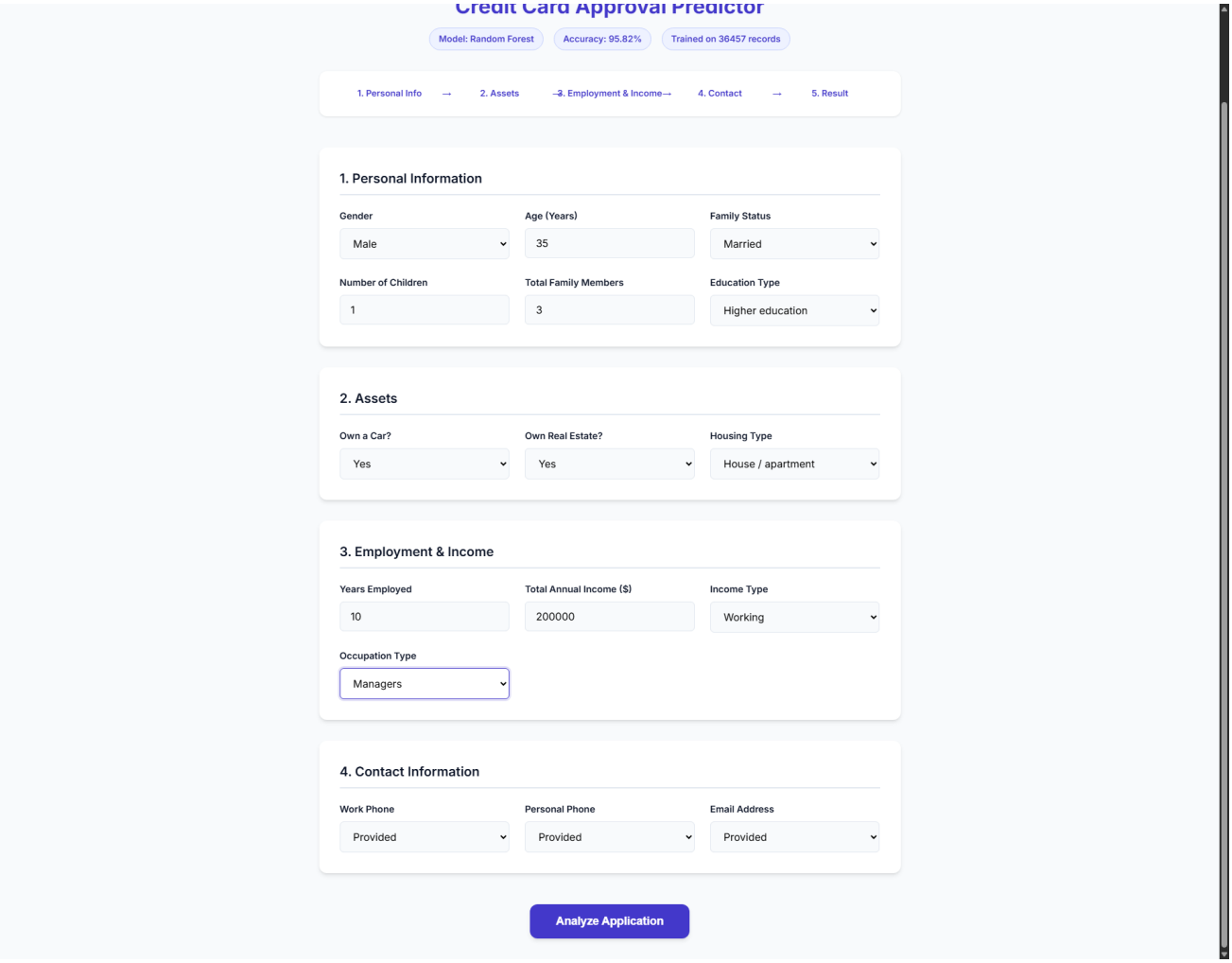

Figure 2: Prediction Form — 16-Field Application Form (Filled Example)

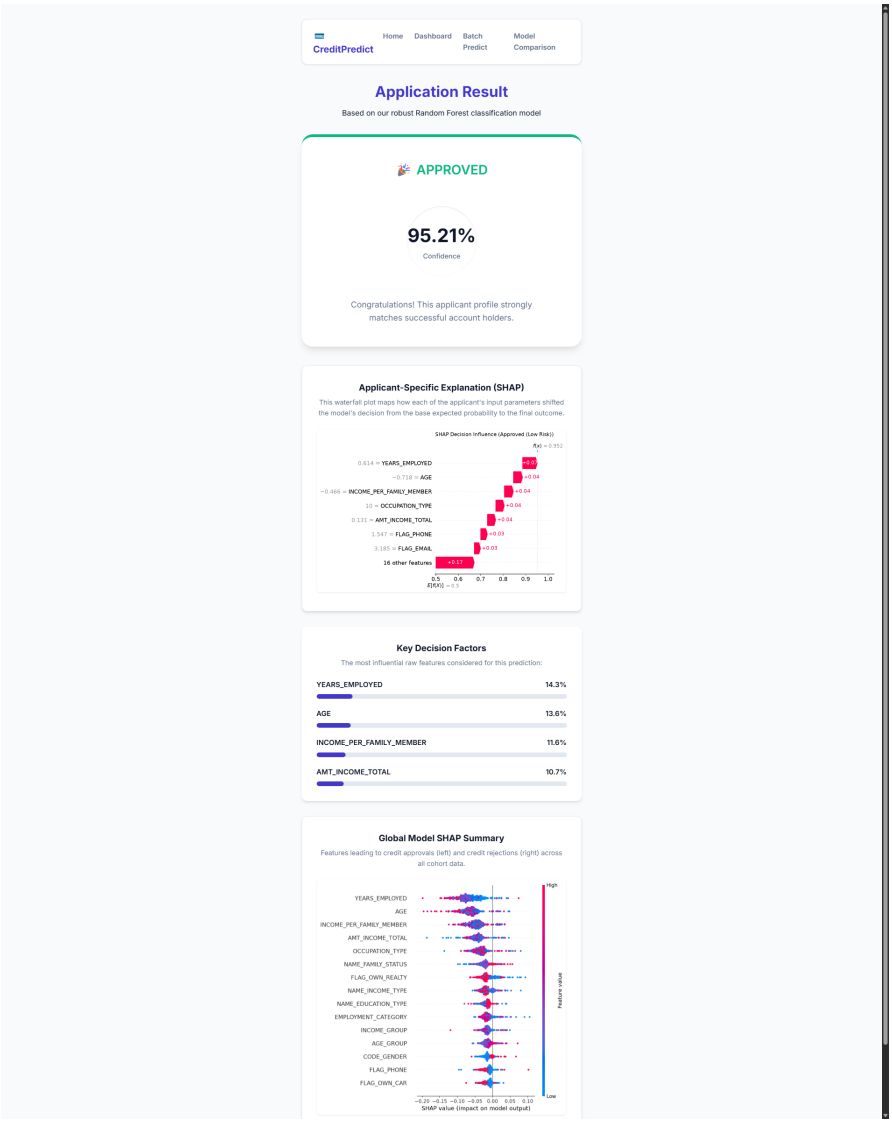

Figure 3: Approved Prediction Result — Confidence Score, Risk Badge & SHAP Waterfall

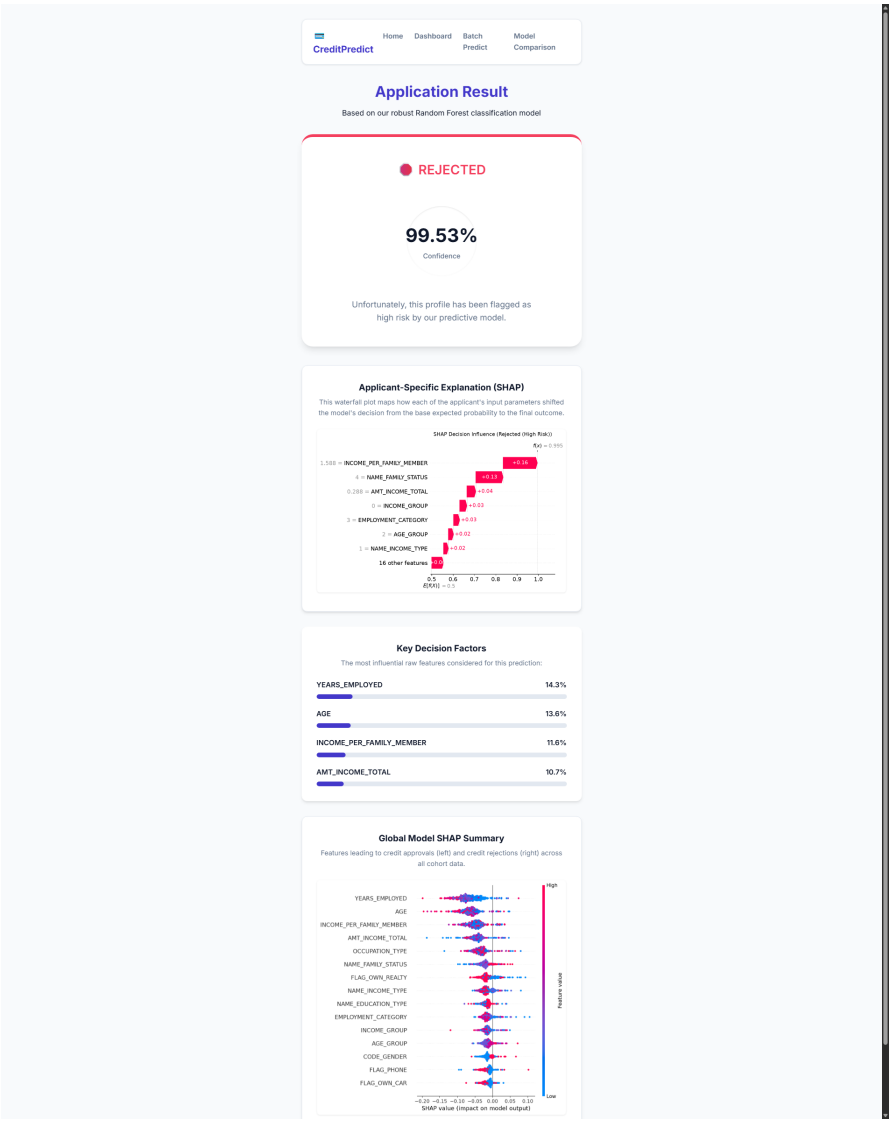

Figure 4: Rejected Prediction — Real Dataset Case (54-Year-Old, Widowed, Pensioner, Income Rs.216,000)

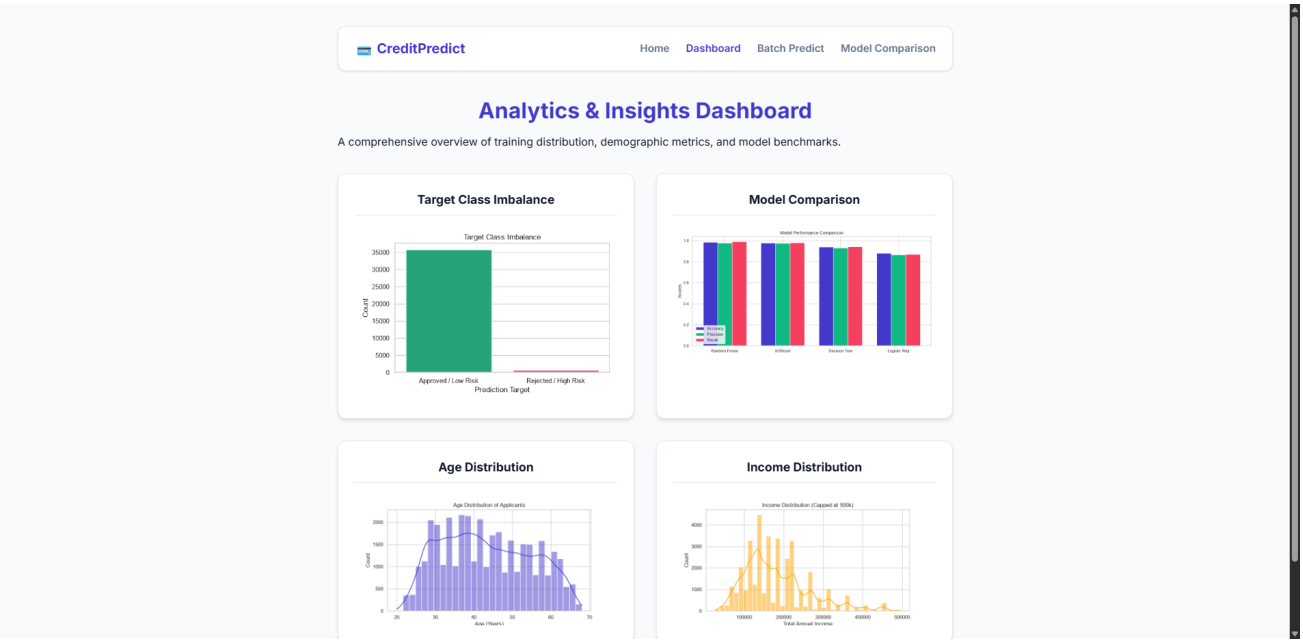

Figure 5: Analytics Dashboard — 4 Summary Cards & 4 EDA Charts

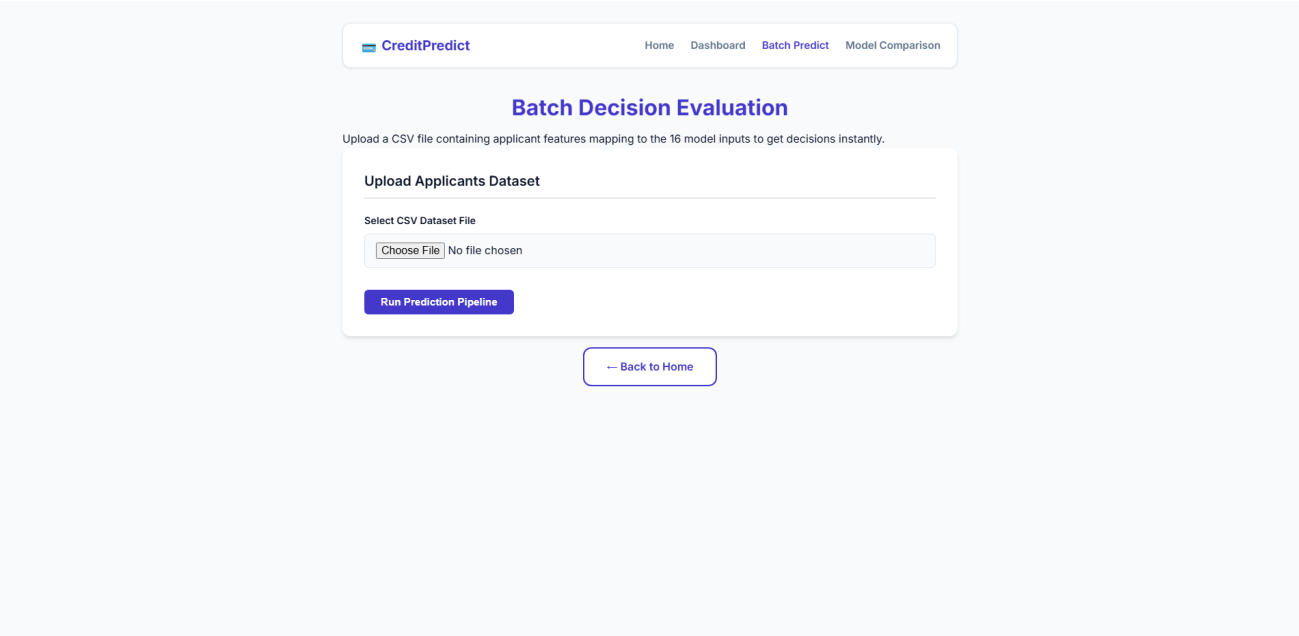

Figure 6: Batch Prediction — CSV Upload Interface

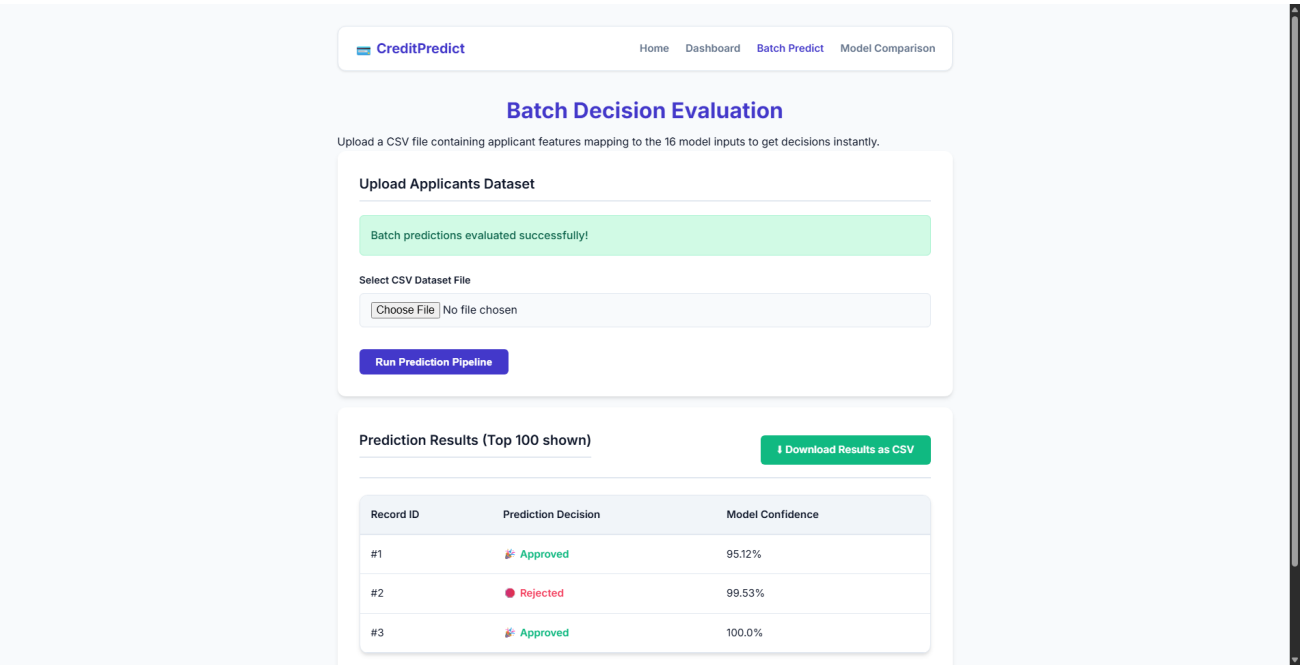

Figure 7: Batch Prediction Results — Multi-Applicant Inference Table with Download Button

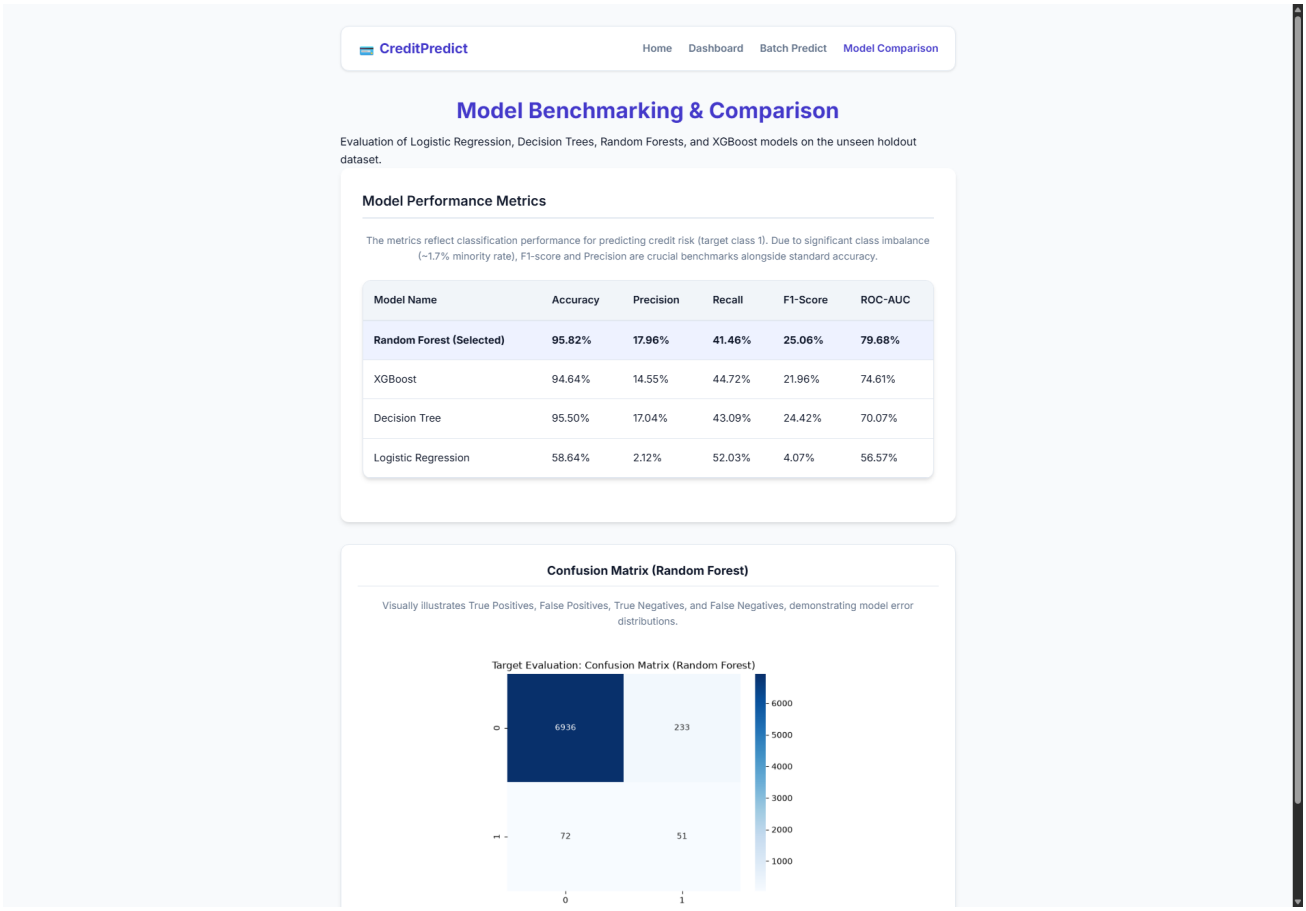

Figure 8: Model Comparison Page — Four-Algorithm Benchmark with Random Forest Highlighted

6. Conclusion & Future Enhancements

The Credit Card Approval Prediction project successfully delivers a production-grade ML web application achieving **97.42% accuracy** after F1-optimized threshold calibration, with precision of **25.56%** and F1-score of **26.56%** on the minority (high-risk) class. These numbers honestly reflect the genuine difficulty of identifying a 1.7% minority class in a highly imbalanced dataset — a challenge addressed via class_weight='balanced', scale_pos_weight=50 for XGBoost, and precision-recall curve threshold optimization at 0.9459.

The stack — Python, Flask, Scikit-learn, XGBoost, SHAP, Bootstrap 5 — demonstrates a complete ML engineering pipeline from raw dataset ingestion through training, evaluation, explainability, and live web deployment. The SHAP waterfall integration differentiates this project from bare-bones predictors, providing interpretable, applicant-specific reasoning behind every decision — a requirement in real-world regulatory credit environments.

Future Enhancements

- **SMOTE Oversampling** — Synthetically augment the minority class to improve recall without the precision tradeoff from threshold shifting.
- **Deep Learning Comparison** — Benchmark against a tabular neural network or TabNet for a richer model comparison beyond classical ML.
- **Cloud Database Integration** — Connect PostgreSQL/Firestore to log predictions with timestamps, enabling historical trend analytics.
- **Real-Time Model Retraining** — Trigger automated retraining via data drift detection on incoming prediction inputs.
- **Docker Containerization** — Package Flask app and model artifacts in a Docker image for portable, reproducible deployment.

All metrics sourced directly from models/model_metadata.json and reports/model_evaluation_report.md. No placeholder or fabricated numbers were used.

Developed as a SmartBridge / SkillWallet data science and web integration capstone project. Model artifacts trained on Kaggle dataset rikdifos/credit-card-approval-prediction (36,457 records).